ASSINGMENT 4: Aggregations and Nested Queries
ATHANASIOS GOURDOMICHALIS

1. Objective

This assignment examines SQL queries again, now placing emphasis on grouping with aggregation and nested queries.

2. Prerequisites

A. We must use the pre-constracted database provided at the following link:
   a) https://www.kaggle.com/datasets/maltegrosse/8-m-spotify-tracks-genre-audio features?select=spotify.sqlite

B. We must add a table containing track ratings. The source file is ***ratings.csv***, located within the assignment folder. It must be appended using the following commands:

```
CREATE TABLE IF NOT EXISTS Ratings (user_id INTEGER, album_id TEXT, rating INTEGER);
.mode csv
.import --skip 1 ratings.csv Ratings
```

C. For fast query execution, we have to add the indexes provided by the DDL commands inside the ***indexes.sql.txt*** file, which is provided alongside the other assignment files.

Assignment Requirements

3. Aggregation Queries with Grouping

Answer the following questions by constructing SQL queries that use aggregate functions combined with grouping. Use the **GROUP BY** and **HAVING** clauses where necessary, and ensure the results are appropriately sorted.

1. The number of albums per year for albums with a popularity index greater than 40. *(Expected attributes: year, albums_per_year)*

2. The number of albums per year for albums that either have a popularity index greater than 40 or are of type 'single'. *(Expected attributes: year, albums_per_year)*

3. The number of albums grouped by album type (album_type) and by year. *(Expected attributes: album_type, year, albums_per_type_year)*

4. For the artist 'ABBA', return the number of albums they have participated in per year. *(Expected attributes: artist, year, albums_per_year)*

5. The most popular album per year, excluding instances where popularity is zero. *(Expected attributes: album, year, max_popularity)*

6. The names of artists who have participated in more than 30 albums.

7. For each user in the Ratings table, create a virtual table (view) containing their average rating and total rating count. *(Expected attributes: user_id, avg_rating, rating_count)*

Note: To extract the year from a date attribute, you can use the ***strftime()*** function. For example: **strftime('%Y', release_date / 1000, 'unixepoch') AS year.**

4. Ranking Queries

8. The top 10 most popular albums based on their popularity index. If there are ties at the 10th position, return only one of them. *(Expected attributes: album_title, popularity)*
(Hint: Combine **ORDER BY** with the **LIMIT** clause.)

   The LIMIT clause in SQL restricts the number of records returned by a query, typically paired with ORDER BY to fetch the "top N" records:

```
SELECT <columns>
FROM <table>
ORDER BY <attributes>
LIMIT N;
```

5. Nested Queries & Set Operators

9. Return the names of the albums that hold the maximum popularity index per year by utilizing a nested query (subquery). The resulting must include the fields **'year'** and **'most_popular_album'**, sorted by year and album name.

10. Return the names (**name**) of artists who have at least one participation in a 'Rock' genre and at least one participation in a 'Blues' genre. (**Hint**: Use the **EXISTS** operator).

11. Answer the previous query using relational set operators (**UNION, INTERSECT, EXCEPT**).

6. Album Recommendations

Assume that a pair of users share **similar interests** when they have assigned identical ratings to the same albums. This collaborative information can be leveraged to generate recommendations: **if User A shares a similar profile with User B,** then albums **highly rated by B that have not yet been interacted with by A can be recommended to A.**

12. Find all pairs of users who have evaluated at least 25 common albums with the exact same rating value.

7. Useful Links:

- **Querying data:** https://www.sqlite.org/lang_select.html
- **Filtering with WHERE:** https://www.sqlite.org/lang_expr.html
- **Sorting with ORDER BY:** https://www.sqlite.org/lang_select.html#orderby
- **Grouping with GROUP BY:** https://www.sqlite.org/lang_select.html#groupby
- **Aggregating data:** https://www.sqlite.org/lang_aggfunc.html
- **Limiting results with LIMIT:** https://www.sqlite.org/lang_select.html#limitoffset
- **Joining tables:** https://www.sqlite.org/lang_select.html#joins
- **Subqueries:** https://www.sqlite.org/lang_select.html#correlated_subqueries
- **Set operations like UNION, INTERSECT, EXCEPT:** https://www.sqlite.org/lang_select.html#compound_select_statements
- **Creating views:** https://www.sqlite.org/lang_createview.html
- **Updating data:** https://www.sqlite.org/lang_update.html

8. Project Files

- ***exercise_5.sql:*** A structured script file containing the 12 sequentially numbered SQL queries.